

Table of Contents

- (RAM, PISO, SIPO, ALU) Lab	2
- Eng. Mohamed Salah Labs	8
- (Gray to Binary to 7-sig) Lab	8
- (Mealy and Moore) Lab	10
- (Clk divider, edge counter, moore FSM, and 7-sig) Lab	13
- System Verilog Lab	16
- STA Lab	18

- (RAM, PISO, SIPO, ALU) Lab

ALU

```
1. module alu6 #(parameter width = 8) (  
2.  
3.         input [width-1:0] in_a,  
4.         input [width-1:0] in_b,  
5.         input [2:0] opcode,  
6.         input en,  
7.         output reg [width-1:0] alu_out,  
8.         output a_is_zero  
9. );  
10.  
11.     always @(*) begin  
12.         if (en) begin  
13.             case (opcode)  
14.                 3'b000: alu_out = in_a + in_b;  
15.                 3'b001: alu_out = in_a - in_b;  
16.                 3'b010: alu_out = in_a & in_b;  
17.                 3'b011: alu_out = in_a ^ in_b;  
18.                 3'b100: alu_out = in_a | in_b;  
19.                 3'b101: alu_out = in_a;  
20.                 default: alu_out = 'b0;  
21.             endcase  
22.         end  
23.         else alu_out = 'b0;  
24.     end  
25.  
26.     assign a_is_zero = (in_a == 0) ? 1 : 0;  
27. endmodule  
28.
```

PISO

```
1. module piso #(parameter width = 20, addr = 8)(  
2.  
3.     input clk,  
4.     input rst_n,  
5.     output reg en,  
6.     input [width-1:0] parallel_in,  
7.     output reg serial_out,  
8.     output reg valid  
9. );  
10.     reg [width-1:0] store;  
11.     reg [4:0] count;  
12.     always @(posedge clk or negedge rst_n) begin  
13.  
14.         if (~rst_n) begin  
15.             serial_out <= 1'b0;  
16.             store <= 'b0;  
17.             valid <= 'b0;  
18.             count <= 'd0;  
19.             en <= 1'b0;  
20.             end  
21.  
22.         else begin  
23.  
24.             if(count) begin  
25.                 en <= 0;  
26.                 valid <= 1;
```

```

27.         serial_out <= store[width-1];
28.         store <= {store[width-2:0], 1'b0};
29.         count <= count -1;
30.     end
31.
32.
33.     else if(en) begin
34.         store <= parallel_in;
35.         count <= 5'd20;
36.         en <= 0;
37.         serial_out <= 1'b0;
38.     end
39.
40.     else begin
41.         valid <= 0;
42.         en <= 1;
43.         serial_out <= 1'b0;
44.     end
45. end
46. end
47.
48.
49. endmodule
50.

```

SIPO

```

1. module sipo #(parameter width = 20)(
2.
3.     input clk,
4.     input rst_n,
5.     input shift_en,
6.     input serial_in,
7.     output reg [width-1:0] parallel_out
8. );
9.     reg [4:0] count;
10.    always @(posedge clk or negedge rst_n) begin
11.
12.        if (~rst_n) begin
13.            parallel_out <= 'b0;
14.            count <= 5'd20;
15.        end
16.
17.        else if (shift_en) begin
18.
19.            parallel_out <= {parallel_out[width-2:0], serial_in};
20.
21.            if (count == 1) count <= 5'd20;
22.            else count <= count - 1;
23.
24.        end
25.
26.
27.
28.    end
29.
30. endmodule
31.

```

RAM

```

1. module ram #(parameter DATA_WIDTH = 20, ADDR_WIDTH = 8) (
2. input clk, wr_en, rst_n,
3. input rd_en,
4. input [ADDR_WIDTH-1:0] addr,
5. input [DATA_WIDTH-1:0] din,
6. output reg [DATA_WIDTH-1:0] dout
7. );
8.
9. reg [DATA_WIDTH-1:0] mem [0:2**ADDR_WIDTH-1];
10. integer i;
11. always @(posedge clk or negedge rst_n) begin
12.     if (!rst_n) begin
13.         for (i = 0; i < 2**ADDR_WIDTH; i = i + 1)
14.             mem[i] <= 'b0;
15.         dout <= 'b0;
16.     end
17.     else begin
18.         if (wr_en)
19.             mem[addr] <= din; // write
20.
21.         if (rd_en)
22.             dout <= mem[addr]; // read
23.     end
24. end
25.
26.
27.
28. endmodule
29.

```

TOP

```

1. module lab_6 #(parameter DATA_WIDTH = 20, ADDR_WIDTH = 8, ALU_WIDTH = 8) (
2.     input clk,
3.     input t_rst_n,
4.     input t_wr_en,
5.     input [ADDR_WIDTH-1:0] address,
6.     input [DATA_WIDTH-1:0] data_in,
7.     output [ALU_WIDTH-1:0] alu_output,
8.     output t_a_is_zero
9. );
10.
11. );
12.     wire [DATA_WIDTH-1:0] ram_2_piso;
13.     wire piso_en_ram;
14.     wire serial;
15.     wire serial_valid;
16.     wire [DATA_WIDTH-1:0] sipo_parallel_out;
17.
18.
19.     ram #(.DATA_WIDTH(DATA_WIDTH), .ADDR_WIDTH(ADDR_WIDTH))
20.     call_ram (
21.         .clk(clk),
22.         .rst_n(t_rst_n),
23.         .wr_en(t_wr_en),
24.         .addr(address),
25.         .din(data_in),
26.         .rd_en(piso_en_ram),
27.         .dout(ram_2_piso)
28.     );
29.
30.

```

```

31.     piso #(.width(DATA_WIDTH), .addr(ADDR_WIDTH))
32.     call_piso (
33.         .clk(clk),
34.         .rst_n(t_rst_n),
35.         .en(piso_en_ram),
36.         .parallel_in(ram_2_piso),
37.         .serial_out(serial),
38.         .valid(serial_valid)
39.     );
40.
41.
42.     sipo #(.width(DATA_WIDTH))
43.     call_sipo (
44.         .clk(clk),
45.         .rst_n(t_rst_n),
46.         .shift_en(serial_valid),
47.         .serial_in(serial),
48.         .parallel_out(sipo_parallel_out)
49.     );
50.
51.
52.     alu6 #(.width(ALU_WIDTH))
53.     call_alu (
54.         .a_is_zero(t_a_is_zero),
55.         .alu_out(alu_output),
56.         .en(sipo_parallel_out[19]),
57.         .opcode(sipo_parallel_out[18:16]),
58.         .in_a(sipo_parallel_out[15:8]),
59.         .in_b(sipo_parallel_out[7:0])
60.     );
61.
62. endmodule
63.

```

TB

```

1. module lab_6_tb();
2.
3.     reg clk_tb;
4.     reg rst_n_tb;
5.     reg t_wr_en_tb;
6.     reg [7:0] address_tb;
7.     reg [19:0] data_in_tb;
8.
9.     wire [7:0] tb_alu_out;
10.    wire tb_a_is_zero;
11.
12.    lab_6 DUT (
13.        .clk(clk_tb),
14.        .t_rst_n(rst_n_tb),
15.        .t_wr_en(t_wr_en_tb),
16.        .address(address_tb),
17.        .data_in(data_in_tb),
18.        .alu_output(tb_alu_out),
19.        .t_a_is_zero(tb_a_is_zero)
20.    );
21.
22.    reg [19:0] test [0:3];
23.    integer j;
24.    integer i;
25.
26.    initial begin

```

```

27.         clk_tb = 0;
28.         forever #5 clk_tb = ~clk_tb;
29.     end
30.
31.     initial begin
32.
33.         randomize_vector(test[0]);
34.         randomize_vector(test[1]);
35.         randomize_vector(test[2]);
36.         randomize_vector(test[3]);
37.
38.         rst_n_tb = 0;
39.         t_wr_en_tb = 0;
40.         address_tb = 0;
41.         data_in_tb = 0;
42.
43.         repeat(2) @(negedge clk_tb);
44.         rst_n_tb = 1;
45.
46.         for (j = 0; j < 4; j = j + 1) begin
47.             write_to_ram(j, test[j]);
48.         end
49.
50.         repeat(25) @(negedge clk_tb);
51.
52.         for (i = 0; i < 4; i = i + 1) begin
53.             address_tb = i;
54.             repeat(21) @(negedge clk_tb);
55.
56.             $display("Address = %d, Vector = %b, ALU Out = %b, A Zero = %b",
57.                 i, test[i], tb_alu_out, tb_a_is_zero);
58.
59.             check(test[i], tb_alu_out, tb_a_is_zero);
60.         end
61.
62.         $finish;
63.
64.     end
65.
66.     task randomize_vector;
67.         output [19:0] random_out;
68.         reg alu_en;
69.         reg [2:0] opcode;
70.         reg [7:0] in_a;
71.         reg [7:0] in_b;
72.
73.         begin
74.             alu_en = 1'b1;
75.             opcode = {$random} % 6;
76.             in_a = $random;
77.             in_b = $random;
78.             random_out = {alu_en, opcode, in_a, in_b};
79.         end
80.     endtask
81.
82.     task write_to_ram;
83.         input [7:0] target_addr;
84.         input [19:0] vector_data;
85.
86.         begin
87.             @(negedge clk_tb);
88.             t_wr_en_tb = 1;
89.             address_tb = target_addr;
90.             data_in_tb = vector_data;
91.

```

```

92.         @(negedge clk_tb);
93.         t_wr_en_tb = 0;
94.     end
95. endtask
96.
97. task check;
98.     input [19:0] vec;
99.     input [7:0] alu;
100.    input zero;
101.
102.    reg [7:0] exp_alu;
103.
104.    begin
105.
106.        if (!vec[19])
107.            exp_alu = 8'b0;
108.        else
109.            case (vec[18:16])
110.                3'b000: exp_alu = vec[15:8] + vec[7:0];
111.                3'b001: exp_alu = vec[15:8] - vec[7:0];
112.                3'b010: exp_alu = vec[15:8] & vec[7:0];
113.                3'b011: exp_alu = vec[15:8] ^ vec[7:0];
114.                3'b100: exp_alu = vec[15:8] | vec[7:0];
115.                3'b101: exp_alu = vec[15:8];
116.                default: exp_alu = 8'b0;
117.            endcase
118.
119.            if (alu == exp_alu && zero == (vec[15:8] == 8'b0))
120.                $display("TRUE");
121.            else
122.                $display("FAIL");
123.
124.        end
125.    endtask
126.
127. endmodule
128.

```

- Eng. Mohamed Salah Labs

- (Gray to Binary to 7-sig) Lab

G to b

```
1. module g2b #(parameter w=4)(
2.     input [w-1:0] gray,
3.     output reg [w-1:0] binary
4. );
5.     integer i;
6. always @(*) begin
7.     binary[w-1] = gray[w-1];
8.     for (i = w-1; i > 0; i = i-1) begin
9.         binary[i-1] = binary[i] ^ gray[i-1];
10.    end
11. end
12. endmodule
13.
14.
15.
```

B to 7sig

```
1. module b_to_7sig #(parameter w=4)(
2.
3.     input [w-1:0] binary,
4.     output reg [6:0] seven_sig
5. );
6.
7.     always @(*) begin
8.
9.         case (binary)
10.            4'b0000: seven_sig = 7'b0111111;
11.            4'b0001: seven_sig = 7'b0000110;
12.            4'b0010: seven_sig = 7'b1011011;
13.            4'b0011: seven_sig = 7'b1001111;
14.            4'b0100: seven_sig = 7'b1100110;
15.            4'b0101: seven_sig = 7'b1101101;
16.            4'b0110: seven_sig = 7'b1111101;
17.            4'b0111: seven_sig = 7'b0000111;
18.            4'b1000: seven_sig = 7'b1111111;
19.            4'b1001: seven_sig = 7'b1101111;
20.            4'b1010: seven_sig = 7'b1101111;
21.            default: seven_sig = 7'b1111111;
22.        endcase
23.    end
24. endmodule
25.
```

TOP

```
1. module seven_sig_top #(parameter w = 4)(
2.     input  [w-1:0] gray_in,
3.     output [6:0]   seven_seg_out
4. );
5.
6.     wire [w-1:0] bin_wire;
7.
8.     g2b #(.w(w)) u_g2b (
9.         .gray(gray_in),
10.        .binary(bin_wire)
11.    );
12.
13.    b_to_7sig #(.w(w)) u_b_to_7sig (
14.        .binary(bin_wire),
15.        .seven_sig(seven_seg_out)
16.    );
17.
18. endmodule
19.
```

TB

```
1. module seven_sig_top_tb();
2.     reg [3:0] gray_in;
3.     wire [6:0] seven_seg_out;
4.     integer i;
5.
6.     seven_sig_top dut (
7.         .gray_in(gray_in),
8.         .seven_seg_out(seven_seg_out)
9.     );
10.
11.     initial begin
12.         $dumpfile("waves.vcd");
13.         $dumpvars(0, seven_sig_top_tb);
14.
15.
16.         for (i = 0; i < 16; i = i + 1) begin
17.             gray_in = i;
18.             #10;
19.
20.             if (seven_seg_out == expected(gray_in))
21.                 $display("TRUE; Gray = %b and O/P = %b", gray_in, seven_seg_out);
22.             else
23.                 $display("FALSE; Gray = %b and expected o/p = %b but o/p is = %b", gray_in,
24.                     expected(gray_in), seven_seg_out);
25.             end
26.             $finish;
27.         end
28.
29.         function [6:0] expected(input [3:0] g);
30.             reg [3:0] b;
31.             begin
32.                 b[3] = g[3];
33.                 b[2] = b[3] ^ g[2];
34.                 b[1] = b[2] ^ g[1];
35.                 b[0] = b[1] ^ g[0];
36.
37.                 case (b)
```

```

37.     4'b0000: expected = 7'b0111111;
38.     4'b0001: expected = 7'b0000110;
39.     4'b0010: expected = 7'b1011011;
40.     4'b0011: expected = 7'b1001111;
41.     4'b0100: expected = 7'b1100110;
42.     4'b0101: expected = 7'b1101101;
43.     4'b0110: expected = 7'b1111101;
44.     4'b0111: expected = 7'b0000111;
45.     4'b1000: expected = 7'b1111111;
46.     4'b1001: expected = 7'b1101111;
47.     default: expected = 7'b1111111;
48.         endcase
49.     end
50. endfunction
51.
52. endmodule
53.

```

- (Mealy and Moore) Lab

Moore and Mealy modules

```

1. module moore (
2.     input clk,
3.     input reset,
4.     input in,
5.     output reg out
6. );
7.
8.     localparam s0 = 2'b00;
9.     localparam s1 = 2'b01;
10.    localparam s2 = 2'b10;
11.
12.    reg [1:0] current, next;
13.
14.
15.    always @(posedge clk or posedge reset) begin
16.        if (reset) current <= s0;
17.        else current <= next;
18.    end
19.
20.    always @(*) begin
21.        case (current)
22.            s0: if (in) next = s1; else next = s0;
23.            s1: if (in) next = s2; else next = s0;
24.            s2: if (in) next = s2; else next = s0;
25.            default: next = s0;
26.        endcase
27.    end
28.
29.    always @(*) begin
30.        if (current == s1) out = 1'b1;
31.        else out = 1'b0;
32.    end
33. endmodule
34.
35.
36.
37. module mealy (
38.     input clk,
39.     input reset,
40.     input in,

```

```

41.     output reg out
42. );
43.
44.     localparam s0 = 1'b0;
45.     localparam s1 = 1'b1;
46.
47.     reg current, next;
48.
49.     always @(posedge clk or posedge reset) begin
50.         if (reset) current <= s0;
51.         else current <= next;
52.     end
53.
54.     always @(*) begin
55.         case (current)
56.             s0: if (in) next = s1;
57.                 else next = s0;
58.             s1: if (in) next = s1;
59.                 else next = s0;
60.
61.             default: next = s0;
62.         endcase
63.     end
64.
65.     always @(*) begin
66.
67.         if (current == s0 && in == 1'b1) out = 1'b1;
68.         else out = 1'b0;
69.     end
70. endmodule
71.

```

Tb

```

1. module tb_edge_detector;
2.
3.     reg clk;
4.     reg reset;
5.     reg in;
6.
7.     wire moore_out;
8.     wire mealy_out;
9.
10.    moore DUT_moore (
11.        .clk(clk),
12.        .reset(reset),
13.        .in(in),
14.        .out(moore_out)
15.    );
16.
17.    mealy DUT_mealy (
18.        .clk(clk),
19.        .reset(reset),
20.        .in(in),
21.        .out(mealy_out)
22.    );
23.
24.    initial begin
25.        clk = 0;
26.        forever #5 clk = ~clk;
27.    end
28.

```

```
29.     initial begin
30.
31.
32.         $dumpfile("waves.vcd");
33.         $dumpvars(0, tb_edge_detector);
34.
35.
36.
37.         reset = 1;
38.         in = 0;
39.
40.         #15 reset = 0;
41.
42.         #12 in = 1;
43.         #10 in = 0;
44.
45.         #20;
46.
47.         #13 in = 1;
48.         #40 in = 0;
49.
50.         #20;
51.         $finish;
52.     end
53.
54.     always @(in or moore_out or mealy_out) begin
55.         $display("At the time %t, the i/p is %b & Moore= %b && Mealy= %b", $time, in, moore_out,
mealy_out);
56.     end
57.
58. endmodule
59.
```

- (Clk divider, edge counter, moore FSM, and 7-sig) Lab

Modules + top:

```
1. module top_system (
2.     input clk_50,
3.     input reset,
4.     input in,
5.     output [6:0] seg_out
6. );
7.
8.     wire clk_half;
9.     wire tick_sig;
10.    wire [3:0] count_sig;
11.
12.
13.    clk_div #(.DIVISOR(4)) div (
14.        .clk_in(clk_50),
15.        .reset(reset),
16.        .clk_out(clk_half)
17.    );
18.
19.    moore moore_tick (
20.        .clk(clk_half),
21.        .reset(reset),
22.        .in(in),
23.        .out(tick_sig)
24.    );
25.
26.    edge_counter count (
27.        .clk(clk_half),
28.        .reset(reset),
29.        .tick(tick_sig),
30.        .count(count_sig)
31.    );
32.
33.    b_to_7sig segment (
34.        .binary(count_sig),
35.        .seven_sig(seg_out)
36.    );
37.
38. endmodule
39.
40.
41. module clk_div #(parameter DIVISOR = 8) (
42.     input wire clk_in,
43.     input wire reset,
44.     output reg clk_out
45. );
46.     localparam MAX_COUNT = (DIVISOR / 2) - 1;
47.     reg [$clog2(DIVISOR)-1:0] count;
48.
49.     always @(posedge clk_in or negedge reset) begin
50.         if (!reset) begin
51.             count <= 0;
52.             clk_out <= 1'b0;
53.         end
54.         else begin
55.             if (count == MAX_COUNT) begin
56.                 count <= 0;
57.                 clk_out <= ~clk_out;
58.             end
59.             else begin
```

```

60.             count <= count + 1'b1;
61.         end
62.     end
63. end
64. endmodule
65.
66.
67.
68. module edge_counter (
69.     input clk,
70.     input reset,
71.     input tick,
72.     output reg [3:0] count
73. );
74.     always @(posedge clk or negedge reset) begin
75.         if (!reset)
76.             count <= 4'b0000;
77.         else if (tick)
78.             count <= count + 1;
79.     end
80. endmodule
81.
82.
83.
84. module moore (
85.     input clk,
86.     input reset,
87.     input in,
88.     output reg out
89. );
90.     localparam s0 = 2'b00;
91.     localparam s1 = 2'b01;
92.     localparam s2 = 2'b10;
93.
94.     reg [1:0] current, next;
95.
96.     always @(posedge clk or negedge reset) begin
97.         if (!reset) current <= s0;
98.         else current <= next;
99.     end
100.
101.     always @(*) begin
102.         case (current)
103.             s0: if (in) next = s1;
104.                 else next = s0;
105.             s1: if (in) next = s2;
106.                 else next = s0;
107.             s2: if (in) next = s2;
108.                 else next = s0;
109.
110.             default: next = s0;
111.         endcase
112.     end
113.
114.     always @(*) begin
115.         if (current == s1) out = 1'b1;
116.         else out = 1'b0;
117.     end
118. endmodule
119.
120.
121. module b_to_7sig #(parameter w=4)(
122.     input [w-1:0] binary,
123.     output reg [6:0] seven_sig
124. );

```

```

125.     always @(*) begin
126.         case (binary)
127.             4'b0000: seven_sig = 7'b0111111;
128.             4'b0001: seven_sig = 7'b0000110;
129.             4'b0010: seven_sig = 7'b1011011;
130.             4'b0011: seven_sig = 7'b1001111;
131.             4'b0100: seven_sig = 7'b1100110;
132.             4'b0101: seven_sig = 7'b1101101;
133.             4'b0110: seven_sig = 7'b1111101;
134.             4'b0111: seven_sig = 7'b0000111;
135.             4'b1000: seven_sig = 7'b1111111;
136.             4'b1001: seven_sig = 7'b1101111;
137.             default: seven_sig = 7'b1111111;
138.         endcase
139.     end
140. endmodule
141.

```

Tb

```

1. module tb_top_system;
2.
3.     reg clk_50;
4.     reg reset;
5.     reg in;
6.     wire [6:0] seg_out;
7.
8.     top_system uut (
9.         .clk_50(clk_50),
10.        .reset(reset),
11.        .in(in),
12.        .seg_out(seg_out)
13.    );
14.
15.    always #10 clk_50 = ~clk_50;
16.
17.    initial begin
18.        clk_50 = 0;
19.        reset = 0;
20.        in = 0;
21.
22.        #20 reset = 1;
23.
24.        #40 in = 1;
25.        #40 in = 0;
26.
27.        #40 in = 1;
28.        #40 in = 1;
29.        #40 in = 0;
30.
31.        #10
32.    end
33.
34. endmodule
35.

```

- System Verilog Lab

```
1. module array_partition;
2.     int arr[] = '{9, 7, 4, 6, 2, 8, 6, 5};
3.     int after[$]; // to be able to use the .pushg
4.
5.     initial begin
6.         foreach (arr[i]) begin
7.
8.             if (arr[i] % 2 == 0) after.push_front(arr[i]); // if even, pushes it to the
front
9.             else after.push_back(arr[i]); // if odd, pushes it to the back
10.        end
11.
12.        $display("after partitioning, arr is %p", after);
13.
14.    end
15. endmodule
16.
17. // O/P: {6, 8, 2, 6, 4, 9, 7, 5}
18.
19. // =====//
20.
21. module max_cons;
22.
23.     int arr[] = '{1, 1, 0, 1, 1, 1, 1, 0, 0, 0, 1};
24.     int count = 1, max_count = 1, num = 0;
25.
26.     initial begin
27.         if (arr.size() > 0) num = arr[0];
28.
29.         for (int i = 1; i < arr.size(); i++) begin
30.             if (arr[i] == arr[i-1]) begin
31.                 count++;
32.                 if (count > max_count) begin
33.                     max_count = count;
34.                     num = arr[i];
35.                 end
36.             end
37.             else count = 1;
38.         end
39.
40.         $display("the max consecutive number is %0d with consecutivity of %0d", num, max_count);
41.     end
42.
43. endmodule
44.
45. // O/P: the max consecutive number is 1 with consecutivity of 4
46.
47. // =====//
48.
49.
50.
51. module second_max;
52.
53.     int arr[] = '{45, 34, 67, 89, 78};
54.     int max1 = -1, max2 = -1; // since we know the array, these initials are ok
55.
56.     initial begin
57.         foreach (arr[i]) begin
58.             if (arr[i] > max1) begin
59.                 max2 = max1;
60.                 max1 = arr[i];
```

```

61.             end
62.             else if (arr[i] > max2 && arr[i] != max1) max2 = arr[i];
63.
64.         end
65.
66.         $display("second max. is %0d", max2);
67.     end
68. endmodule
69.
70. // second max. is 78
71.
72. // =====//
73.
74. module freq_counter;
75.     int arr[] = '{8, 3, 3, 4, 5, 6, 3, 5, 4, 6, 8, 7, 6, 4, 3, 5, 6}';
76.
77.     int count[10]; // this will contain the counts for each num, where the index represents
the num
78.
79.     initial begin
80.         for (int i = 0; i < arr.size(); i++) begin
81.             count[arr[i]]++; // the increments the index of the num in the "count"
82.         end
83.
84.         for (int num = 0; num < 10; num++) begin
85.             if (count[num] > 0) $display("%d --> %d", num, count[num]);
86.         end
87.     end
88. endmodule
89.
90. /*
91. O/P:
92.
93. # KERNEL:          3 -->          4
94. # KERNEL:          4 -->          3
95. # KERNEL:          5 -->          3
96. # KERNEL:          6 -->          4
97. # KERNEL:          7 -->          1
98. # KERNEL:          8 -->          2
99. */
100.
101.
102.
103.

```

- STA Lab

الموضوع: التاريخ: ←

→ STA Lab

✗ • Setup = available - arrival

$$T_{\text{avail}} = T_{\text{ck}} + T_{\text{skew}} + T_{\text{jitter}}$$

$$T_{\text{arr}} = T_{\text{cq}} + T_{\text{wire}} + T_{\text{comb}}$$

Givens

→ $T_{\text{ck}} = \frac{1}{40\text{MHz}} = 25\text{ns}$

→ FF1 clk = $6\text{ns} = 2+2+2$

→ FF2 clk = $9\text{ns} = 2+2+2.5+2.5$

Setup & $T_{\text{arr}} = 25 + 3 - 2 = 26\text{ns}$

for Path 1 → $T_{\text{arr}} = 6 + 5 + 12 + 4 = 27\text{ns}$

∴ slack = $26 - 27 = -1\text{ns}$ V.IOLATION

for Path 2 → $T_{\text{arr}} = 6 + 2 + 13 + 4 = 25\text{ns}$

∴ slack = $26 - 25 = 1\text{ns}$ → no violation

✗ • hold = $T_{\text{arr}} - T_{\text{req}}$

$$T_{\text{arr}} = T_{\text{cq}} + T_{\text{wire}} + T_{\text{comb}}$$

$$T_{\text{req}} = T_{\text{skew}} + T_{\text{hold}} + T_{\text{jitter}}$$

∴ $T_{\text{req}} = 3 + 3 + 2 = 8\text{ns}$ no violation

for Path 1: $T_{\text{arr}} = 2 + 5 + 2 = 9$; ∴ hold = $9 - 8 = 1\text{ns}$

for Path 2: $T_{\text{arr}} = 2 + 2 + 1 = 5$; ∴ hold = $5 - 8 = -3\text{ns}$ violation

- To solve the setup violation: Reduce clock frequency
- To solve the hold violation: use delay buffers in the fast data path